



Novel hybrid deep learning based cyber security threat detection model with optimization algorithm

S. Markkandeyan, A. Dennis Ananth, M. Rajakumaran*, R.G. Gokila, R. Venkatesan, B. Lakshmi

School of Computing, SASTRA Deemed University, Thanjavur, Tamilnadu, India

ARTICLE INFO

Keywords:

Internet of things (IoT)
Cyber-attacks
Security
Adaptive tensor flow deep neural network
Improved particle swarm optimization (IPSO)
Enhanced long short term memory (E-LSTM)

ABSTRACT

In order to continuously provide services to the company, the Internet of Things (IoT) connects the hardware, software, storing data, and applications that could be utilized as a new port of entry for cyber-attacks. The privacy of IoT is presently very vulnerable to virus threats and software piracy. Threats like this have the potential to capture critical data, harming businesses' finances and reputations. We have suggested a hybrid Deep Learning (DL) strategy in this study to identify malware-infected programs and files that have been illegally distributed over the IoT environment. To detect illegal content utilizing Source code (SC) duplication, the Adaptive TensorFlow deep neural network with Improved Particle Swarm Optimization (IPSO) is suggested. This novel hybrid strategy improves cyber security by fusing cutting-edge DL with optimization methods, providing more effective and accurate detection. With a strong solution for real-time threat identification, the model handles the complexity of contemporary cyberthreats. To highlight the significance of the proxy regarding the SC duplication, the noisy data is filtered using the tokenization and weighting feature approaches. After that, duplication in SC is found using a DL method. To look into software piracy, the dataset was gathered via Google Code Jam (GCJ). Additionally, using the visual representation of color images, the Enhanced Long Short-Term Memory (E-LSTM) was employed to identify suspicious actions in the IoT environment. The Maling dataset is used to gather the malware samples required for testing. The experimental findings show that, in terms of categorization, the suggested method for evaluating cybersecurity threats in IoT surpasses conventional approaches.

1. Introduction

IT systems are becoming more complicated as daily data output rates increase. Human monitoring is not a realistic option because the human brain is unable to discriminate between data that contains malware or viruses and conventional data. Cybersecurity is one of those fields that use both ML algorithms and human experience to distinguish between anomalies. It can be described as the junction of information security, network security, and computer security Dalal and Rele (2018). To stop system intrusion, cybersecurity has become more crucial. Due to the introduction of new methods of infiltration that make advantage of operating system weaknesses and network communication settings, among other things, previously, a firewall's security settings wouldn't have been enough to stop these intrusions Sornsuwit and Jaiyen [1,2]. With the improvement of artificial intelligence (AI) capabilities, learning based systems for recognizing cyber-attacks have become more sophisticated, and they have produced significant outcomes in a variety of studies.

However, protecting IT systems from the threats and criminal network behavior is still very difficult because cyber-attacks are al-

ways changing. Due to regular network intrusions and harmful actions, effective defenses and security concerns were given key importance for developing trustworthy solutions. For identifying network breaches and cyber threats, there are typically two main systems Lee et al. [3] and Dalal et al. [4]. It is essential for protecting intellectual property, stopping software piracy, and preserving the integrity of software systems to detect illegal content through SC duplication. It assists in detecting the illicit use of proprietary code, guaranteeing adherence to legal requirements, and preserving the confidentiality of digital assets. Maintaining moral behavior and lowering the dangers of code theft and illicit software development depends on this identification. Types of cyber security assaults are depicted in Fig. 1.

Most currently employed techniques for recognizing cyber attacks match potential attack characteristics by blocking harmful connections to assist defenders in attack scenario analysis. Typically, malicious attempts to compromise network breaches use phishing websites or operating system updates with legitimate network protocols like HTTP, ICMP, and SSL to get past firewalls and virus detection engines and al-

Peer review under responsibility of KeAi Communications Co., Ltd.

* Corresponding author.

E-mail address: mrajakumaran90@gmail.com (M. Rajakumaran).

<https://doi.org/10.1016/j.csa.2024.100075>

Received 26 June 2024; Received in revised form 17 September 2024; Accepted 19 October 2024

Available online 20 October 2024

2772-9184/© 2024 The Authors. Publishing Services by Elsevier B.V. on behalf of KeAi Communications Co., Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

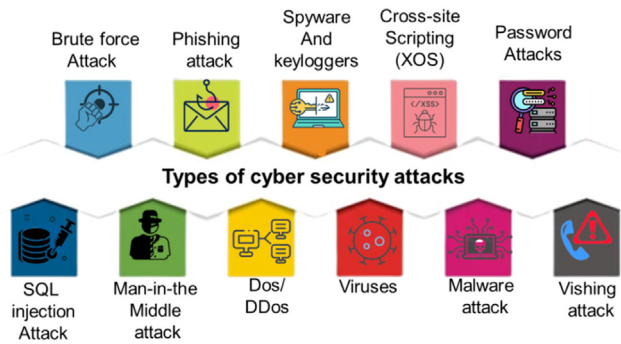


Fig. 1. Cybersecurity attack types.

low users to download malicious applications into hosts that were under the control of remote controllers Lin et al. [5].

The following factors are used to determine the cybersecurity level: accessibility, data aggregation, protection, preservation, and transport of information through digital and other methods. Cybersecurity highlights three important factors. The data, how it is processed and communicated, the physical and virtual settings in which it is used, the degree of protection achieved through the employment of such measures, as well as the relevant professional concerns. The term cybersecurity refers to a method for preventing unauthorized access to, use of, disclosure of, modification of, or destruction of computer systems, networks, and data, according to our definition. Cybersecurity has been defined in the context of this study as the body of guidelines, safeguards, risk management techniques, policies, practices, technologies, processes, and training that can be applied to protect a company's online infrastructure as well as user assets Thakur et al. [6].

1.1. Contributions

The main issue in cyber security threat detection is the challenge of recognizing advanced persistent threats (APTs). APTs are complex and are planned endurance attacks, whose main goal is to being in a network and remain there for an extended duration without being discovered. These threats are normally concealed through fileless malware, zero-day vulnerability, or concealment of the communication channel, hence making it challenging to identify them through a firewall or traditional antivirus scanner. Towards this, we have proposed a combined ATFDNN+IPSO approach in this study to detect those programs and files which have been infected by malware and are being illegitimately shared in the IoT context. For the purpose of identifying Illegal content using Source code (SC) duplication, the Adaptive TensorFlow deep neural network with Improved Particle Swarm Optimization (IPSO) model is put forward. This new mixed strategy enhances the cyber security since it employs DL the most advanced method in machine learning alongside optimization techniques yielding better and accurate result. The model provides a reliable solution for threat detection in real time, which allows effectively solving the challenges of modern threats. The optimization of the IPSO improves the functionality of the Adaptive TensorFlow deep neural network to facilitate better algorithms' accuracy, particularly in detecting the complex threats.

The format for the remaining portions of the essay can be seen here. Section 2 reviews the related works, Section 3 describes this approach to be used, Section 4 presents the experimental setup and discusses the outcomes of the results, and Section 5 provides concluding observations.

2. Related works

Ullah et al. [7] outlined a hybrid DL approach for checking the entire IoT infrastructure for viruses and illicitly downloaded software. To find copied SC in unlawfully distributed software, a Tensor Flow deep neural network is suggested. Through the use of tokenization and weighting

feature techniques, the data noise is reduced and the significance of each token in terms of SC plagiarism is increased. After that, the problem of detecting plagiarism in SC is handled using the DL approach. Barik et al. [8] provide the DL techniques, datasets, and comparative analysis to detect cyber security intrusions. To be more specific, we examine the state of intrusion detection systems that employ DL techniques. Karie et al. [9] examine the prior literature and present recent improvements in IoT threat detection study, concentrating on the numerous IoT security issues and the most recent methods for fending off cyber security dangers in IoT networks. Sarker et al. [10] introduced a tree for identifying invasions. To do this, the Machine Learning (ML) based security paradigm is employed.

The priority rankings of the security factors are taken into consideration by IntruDTree before using these characteristics to develop a tree-based generalized intrusion detection model. Le et al. [11] proclaimed the creation of IMIDS, a smart intrusion detection system (IDS) designed to shield IoT devices from hackers. To classify various forms of cyber threats, IMIDS relies on a lightweight convolutional neural network (CNN) model. Yin et al. [12] introduced a novel approach to vulnerability evaluation and attack modeling in DNP3 by combining the parsed DNP3 protocol with extra data, such as malware samples, and utilizing ML to pick features. In addition, we implemented a categorization and visualization procedure into our cyberattack algorithm. Kilincer et al. [13] evaluated previous research on the widely utilized NSL-KDD, UNSW-NB15, ISCX-2012, CSE-CIC IDS-2018, and CIDDS-001 data sets in order to construct IDS systems. Alqahtani et al. [14] evaluate previous research on the widely utilized NSL-KDD, UNSW-NB15, ISCX-2012, CSE-CIC IDS-2018 and CIDDS-001 data sets in order to construct IDS systems.

Kandhro et al. [15] offered a unique approach to cyber-physical system vulnerability and breach detection based on DL. A generative adversarial network is introduced in this study to identify cyberthreats in IoT-driven IICs networks. Aldhyani and Alkahtani [16] analyzed the proposed model to provide Agriculture 4.0 with the highest levels of security against any cyber threats. The Unsupervised Hunting of Anomalous Commands (UHAC) method is built on ML. Kayhan et al. [17] Propose using Security Information and Event Management (SIEM) logs to look for unusual text-based commands that might be used for threat hunting. Sewak et al. (2022) conducted a detailed analysis of the numerous deep reinforcement learning applications in cybersecurity threat assessment as well as defense. To make these systems secure, the emphasis is on employing ML techniques to analyze cybersecurity data (Ahsan et al. [18]. Al Razib et al. [19] and Laso et al. [20] suggested an intrusion detection system (IDS) that uses Software Defined Networking (SDN) and DL to battle new cyber threats in the Internet of Things. Laso et al. (2022) developed a study of the current cyber dangers at sea, concentrating on cruising ships in particular.

2.1. Problem statement

The system is given a set of samples as input. Each sample in the training set is labeled from X_1 to $X_1, X_2, \dots, X_n$, and the training set is marked as X . Each sample is classified as malicious or benign based on its unique properties and given the name $Y_1, Y_2, \dots, Y_n$. The available data is trained using any ML method, like as Support Vector Machines (SVM), Decision Trees (DT), Neural Networks, etc., during the training phase. By calculating the percentage of correctly classified samples, the optimal model among a group of algorithms is found. As a result, training may be thought of as discovering the optimal features for categorizing the data samples. A model is used for the detection of new samples after being trained.

3. Proposed method

Fig. 2 illustrates the architecture paradigm we suggest in this study for industrial IoT cybersecurity risks and defenses. For the purpose of

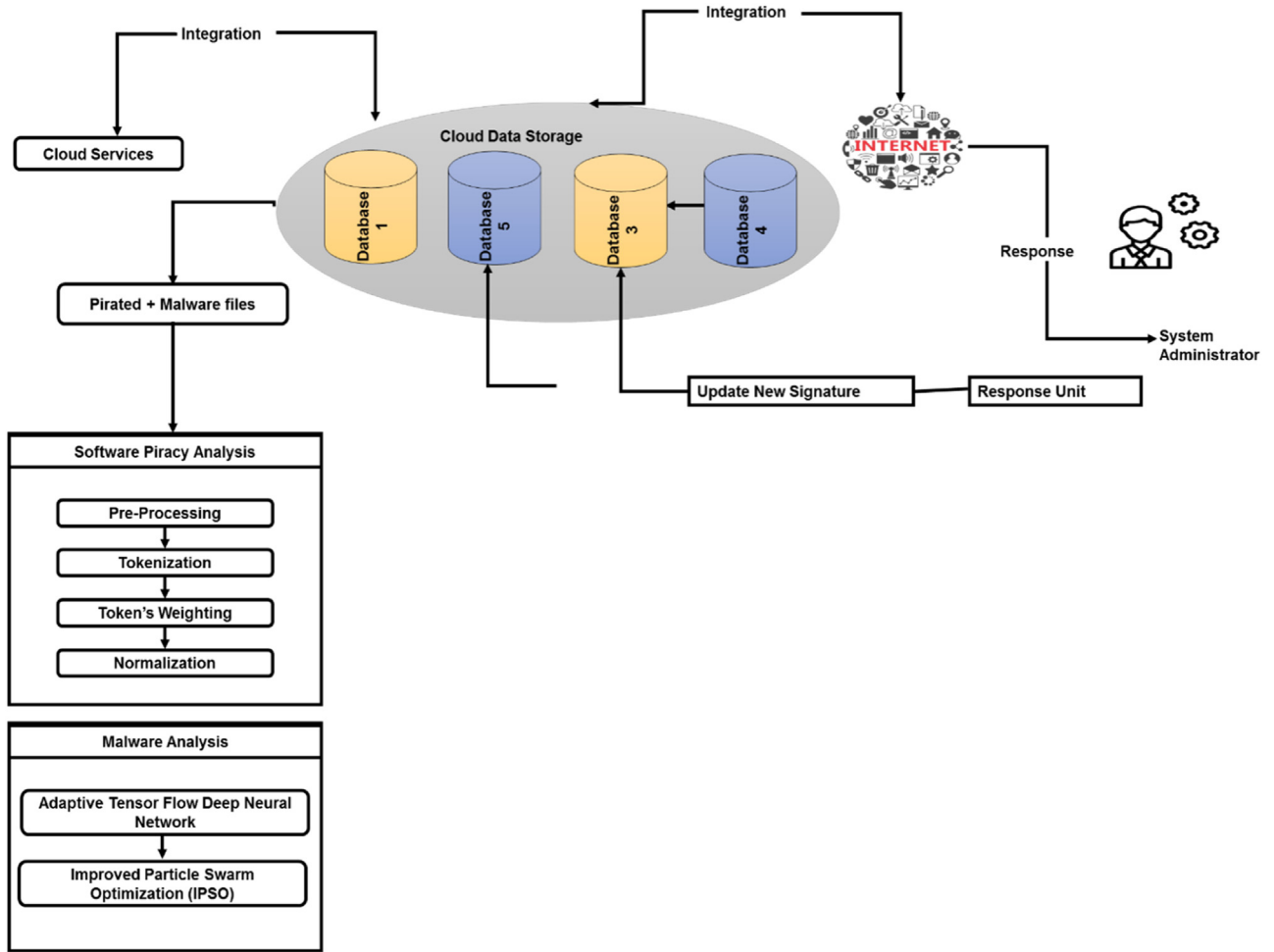


Fig. 2. Block diagram of the proposed method.

managing malware binaries and software piracy files, four databases have been constructed in cloud storage. Raw network traffic data are kept in database 1, whereas historical malware data are kept in database 2. The third database also maintains recent signatures of malware attacks that have just been identified. The cracker stores the illegal software through the IoT devices in the database 4. It serves as a storage area for the illegal copies that crackers attempt to distribute around the IoT network. Processing this much data will be expensive and time-consuming. The pre-processing module received the raw data from the first database. Preprocessing is done on the raw data, and important details are logged. The detection module is then provided the previously processed data. For the purpose of finding malware and other dangers like piracy, the module for detection learns from the signatures in databases 2 and 4. The proposed system alerts the administrator for appropriate action if any harmful behavior is seen in the network.

3.1. Data set

The Maling dataset is a reference data collection that includes pictures of various types of malware. A total of 9939 samples of 25 distinct families of malware are included in the collection's grayscale photos. The binary representation of the malware files was used to make these images. Disparities in data distribution pose a difficulty for classification. From 64 pixels by 200 pixels up to 800 pixels by 800 pixels, the sizes of the images were all over the place. The graph below shows how the collected picture data was distributed among households. A common split ratio in studies was 80:20, where 80% of the data was utilized for

training while 20% for testing. The irregular distribution and volume of the data made this an especially challenging undertaking.

The suggested DL system's main goal is to identify software that has been illegally obtained from a variety of SCs. Many different types of SC can be recognized for plagiarism using DL technology. Fig. 2 illustrates how the cracked, stolen copy of the software made advantage of the logic of the original application. Preprocessing techniques tokenize the SCs at first to minimize the dimensionality of the data and gather crucial information for the next step. To identify plagiarism among various kinds of SCs, the important attributes that have been acquired are used in the Tensor Flow framework and Keras API DL algorithm. A total connection of 400 unique SC documents written by 100 individual programmers makes up the GCJ dataset. The data collection from Pant and Bista [21] is compiled from the Google Code Jam (GCJ) database at Google.

3.2. Data preprocessing

To change the virus detection problem using the image classification issue, raw binary files are turned into color images. It sets the proposed study apart from the newest technique, which turns malware data file together into 256-color image pixels. This approach eliminates the requirement for reverse engineering tools like decompilers and disassemblers. Color photographs can extract more precise information than grayscale images with only 256 hues can.

Additionally, malware graphics with superior attributes can perform better when categorizing malware families. Previously, several ML-

based virus detection techniques produced superior results when used with grayscale photos. The grayscale visualization of the color photos is used with feature extraction techniques to categorize the type of malware. Utilizing feature reduction techniques, the classification performance is enhanced by reducing the number of features. The outcomes showed that using ML techniques is not a better method for identifying viruses because, when used with colored photos, they result in exponential values. Because DL algorithms can automatically apply filters to reduce noise in sizable malware datasets, they perform better. As a result, color images that use DL techniques produce better results.

A malware binary file must go through four phases to become a color image. The raw binary data is converted into the hexadecimal strings first (0–15). Second, an 8-bit vector is used to split a hexadecimal stream into 8 segments, with the next segment being calculated as an unsigned integer (0 – 255). Third, a two-dimensional matrix space is created from the 8-bit vector. Fourth, a red, green, and blue-shaded plot of each 8-bit integer formed in two dimensions is produced. The whole phases of the data preparation portion are shown in Fig. 2.

3.3. Feature selection and feature extraction

It could be challenging to identify pirated software among different source code (SC) types because each one has a different syntax and semantic structure. For the SC similarity detection, we utilized the software plagiarism model. The SCs are divided into manageable chunks for in-depth study utilizing the preprocessing techniques. Stemming, root word extraction, frequency extraction, and word stopping are all included. It removes noisy data and transforms the codes into the useful information. The data is cleaned by removing unwanted information, such as special symbols, stop words, and constants. The cleaned data is then converted into usable tokens through the tokenization process. A malware binary file must go through four phases to become a color image. The original binary data is first transformed into hexadecimal strings (0–15). The second step is to divide a hexadecimal stream into 8 segments, each of which is measured as an unsigned integer using an 8-bit vector (0–255). Eq. (1) illustrates this mathematical definition of TFIDF.

$$tfidf(t, d, D) = if(t, d) \times idf(t, D) \quad (1)$$

Where t stands for the token, d for each document, f for Frequency and D for all the documents utilized in the data collection.

3.4. Adaptive tensor flow deep neural network (ATFDNN) with improved particle swarm optimization (IPSO)

An adaptive particle swarm optimization (APSO) is provided as an alternative to the traditional particle swarm optimization (PSO). The fact that it has a quicker convergence time and can conduct a global search across the entire search space is more relevant. In the APSO, there are two main steps.

3.4.1. Adaptive tensor flow deep neural network

This approach is recommended to analyze malware data thoroughly utilizing Deep Neural Network technology (DNN). The five components of the DNN are shown in Fig. 2. The input layer of the created neural network model receives photos used for training. The first layer used to lower noise and improve signal quality is the convolution layer. The convolutional kernel width, hidden unit count, and learning rate of the proposed DL model are all optimized for maximum accuracy. The last step is to keep crucial data while minimizing data overhead by using a pooling layer. The Two Dimensional (2D) array is reduced to a One Dimensional (1D) representation in the third phase by a fully linked layer, which is subsequently fed into the specific classifier. To identify which malware families are present in the pertinent images, the classifier is applied a fourth time.

An ML system called TensorFlow is utilized for sophisticated calculations in challenging situations. We introduce a novel Tensor Flow API that enables us to build a variety of DL and ML methods (API). It possesses a variety of layer types that may be configured for training of the data, complex calculations, and observing the state of each function's operation. The in-depth learning approach searches multiple programming languages' SCs for similarity using the TensorFlow framework. The stolen coder's identity is then determined employing the retrieved similar codes. The DL model receives the grade values as input. The thick layer, also known as the fully connected layer, is set up for the input and output data. Each of the 3 basic tiers has 100, 50, and 30 neurons, respectively. Input data with a shape parameter and variable are sent to the first layer. As a result of each neuron receiving information from prior levels, they are all extremely interconnected. Using the output variable in the fourth dense layer, the copied code is targeted.

The dropout layer is used to improve DL when optimizer, loss function and activation, and learning error rate are taken into account. The dropout layer also deals with the issue of overfitting. For input variables, the Rectifier (ReLU) activation method is utilized to identify the patterns of incoming data. Eq. (2) contains a mathematical representation of it as the portion of its argument that is positive.

$$f(y) = y^+ = \max(0, y) \quad (2)$$

Where y is the input that the relevant neurons. A sigmoid technique is a successful logistic solution to the multiclass problem. In Eq. (3), it is theoretically defined.

$$S(y) = \frac{1}{1 + e^{-y}} \quad (3)$$

Where S describes the function of sigmoid. The stochastic descent gradient, commonly known as the Adam optimizer, is used to build and improve DL models. Eq. (4) along with Eq. (5) displays the fading means of the squared gradients.

$$n_t = \beta_1 n_{t-1} + (1 - \beta_1) g_t \quad (4)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (5)$$

The estimated means of the first and second instant slopes are, respectively, n_t as well as asv_t . These formulas are used by the Adam optimization method to determine the average of predecessor and successor moments. They serve as the projections for updating the baseline gradient n_t as well as square gradient V_t , running exponential averages. The following contributions are made by the Tensor flow-based algorithm:

- It incorporates numerous types of computational APIs such as the GitHub framework is to create and expand ML methods for large-scale data sets.
- Information, concealed, and softmax layer with a variety of activation functions are used in this method to automatically train the model.
- It extends and updates the intended model while offering solid services.
- Small devices and large networks can be configured and used with these kinds of algorithms.

3.4.2. Improved particle swarm optimization

The multi-swarm scheduling module is proposed with improved particle swarm optimization to hold numerous swarms. Each swarm autonomously manages numerous factors, including the iteration process, position updates, velocity updates, and more. After selecting different periods from the computing environment, the multi-swarm scheduling module receives the most recent findings from each swarm and assesses if they have an impact on other swarms. The scheduling module keeps an eye on each sub-swarm (SS) and collects its output.

There are many particles in each SS. The SS may receive instructions or data from the multi-swarm scheduler, and vice versa. If the present

SS meets the specified requirement, the multi-swarm scheduler initially receives the results that correspond to the p_{best} (local best fitness) and g_{best} (global best fitness) values. These results are only conveyed by the current swarm if $S_i = 1$; otherwise, no records with the p_{best} and g_{best} values are sent.

In Eq. (6), d stands for a threshold together with the maximum and current iteration numbers. $rand()$ provides a uniformly distributed random number (0, 1).

$$S_j = \begin{cases} 1, & \text{if } id_i < \frac{tit_i - it_i}{tit_i} \times rand() \times \text{Fitness} \\ 0, & \text{if } id_i < \frac{tit_i - it_i}{tit_i} \times rand() \times \text{Fitness} \end{cases} \quad (6)$$

3.4.2.1. The requested rule of the multi-swarm scheduler (MSS). The MSS contacts the active SS to request information when necessary. The MSS will broadcast the rule along with $n = \{1, 2, \dots, 100\}$ if the SS, where $k = 3$ has transmitted the swarm request rules (SRR) more than $thek$ times. Finding the sub-activity swarm's current level is the first step in the MSS request rule. As the SS becomes more active, it becomes more beneficial since it might result in the best outcomes.

3.4.2.2. The multiple-swarm gathering principle. The multi-swarm scheduler uses data from a storage table to update p_{best} and g_{best} using information from the alive SS.

3.4.2.3. The formula for eliminating several swarms. Eq. (7) states that a swarm will be terminated if it broadcasts the SRR k times along with the MSS decide to remove the present SS. The present SS is terminated by the MSS if the g_{best} is not changed in pn iterations. We initialized PSO and set pn . Eq. (7) uses the $ite(l)$ function to calculate the living sub-size, swarm's m , a threshold, and pl , the number of times the SS communicates the SRR.

$$f_i = \frac{\sum_{l=1}^n ite(l) \times m}{pl} \quad (7)$$

If the swarm does not alter the g_{best} in pn iterations, the MSS removes the current SS. During PSO's initialization, we set pn .

3.4.3. IPSO algorithm.

- **Stage 1:** Change the dataset's format from stream to object after loading it from the text file. For the initialization of PSO, accumulate the formatted memory data in the temporary table. Randomize swarm size initialization, and then allocate varying amounts of RAM to each swarm. Calculate the objective function after initializing all particle coordinates X_{ij} and velocities V_{ij} with random values. Each swarm's p_{best} (local best) and g_{best} (global best) values need to be updated. Achieve Stage 2.
- **Stage 2:** Include information on each swarm's parameters, including particle size, iterations, lower and upper constraints on velocity, C_1 and C_2 from Eq. (12), d_i from the multi-swarm destruction rule, m from the learning factor of cognition, and pn from the learning factor of social interaction in Eq. (12). Set the current particle number = 1, the iteration number = 0, tit_i = size of the particles to be used, and it_i = iteration number to 0. Achieve Stage 3.
- **Stage 3:** If there are no changes for <45 iterations in the iteration number, current iteration number, or g_{best} , then each swarm executes stage 4; otherwise, the swarm is destroyed and stage 10 is carried out. By comparing the current swarm's g_{best} with the previous one, the primary scheduling module updates the p_{best} and decides whether to employ the MSS request rule to update the g_{best} . Run the multi-swarm collection rule if g_{best} or p_{best} are modified.
- **Stage 4:** Move to Stage 5 if the current particle number in each swarm equals the particle size; if not, proceed to Stage 9.
- **Stage 5:** Each particle alters its position and speed, and each swarm receives g_{best} and p_{best} values. Stage 6 is next.

- **Stage 6:** Limit each person's movement in space and time. Stage 7 is next.
- **Stage 7:** Each particle's fitness is assessed, and p_{best} and g_{best} are updated. Execute the SRR and then move on to the Stage 8. If the present swarm needs to be eliminated, do so and depart while adhering to the multi-swarm destroying rule.
- **Stage 8:** Current particle number = current particle number + 1. Go to Stage 4.
- **Stage 9:** Current iteration number = current iteration number + 1. Go to Stage 3.
- **Stage 10:** Before leaving, the multi-swarm collection rule ought to be executed.

3.4. Enhanced long-short term memory

3.4.1. Long short-term memory (LSTM)

Fig. 3 illustrates how the Hidden Layer (HL) node in the recurrent neural network is taken by the memory block sub-network in the LSTM model (RNN).

A memory cell, a Forget Gate (FG), input and output squashing units, input and output gate make up one memory block. The list of memory block components is presented below. The input squashing unit utilized nonlinear transformation to handle this input x_t at time step t and the memory block output h_{t-1} at time step $t - 1$. They continue through an activation function, which causes the input to be squashed.

The input gate receives the weighted sum of and through a sigmoid function, which is then used to generate. The input gating is built upon the Hadamard product of that and. As a result, the information flow through the input squashing unit pauses and then resumes when the input gate outputs are, respectively, 0 and 1. A distinguishing quality of the LSTM model is its use of multiplicative gates.

The memory cell with the self-connected recurrent edge serves as the focal point of the memory block. It is updated via a continuous linear function. To prevent evaporation or blowing up, the gradient must remain constant over a large number of time steps. This calls for the self-connected, constant-weight, recurrent edge 1. The state that a memory cell is in right now is its internal state. The symbol for the Hadamard product in vector notation is $S_t = s_{t-1} + i_t \otimes g_t$.

The weighted average of x_t as well as h_{t-1} is transferred via the forget gate, which then f_t generates using a sigmoid function. The contents are flushed by the internal state f_t . Particularly helpful in networks that run continuously is this design $S_t = s_{t-1} + i_t \otimes g_t$. The weighted sum of x_t and h_{t-1} is created and then, using a sigmoid function, sent to the output gate o_t . The output squashing unit generates a $\tanh s_t$ function to transmit the internal state (s_t). The output of the memory block h_t is the Hadamard product of o_t and the squashing $\tanh(s_t)$. The Hadamard product of the squashing $\tanh(s_t)$ is the output of the memory block h_t .

3.4.2. Enhanced LSTM (E-LSTM)

In order to satisfy the needs for training complex retrieval and computing resources, as well as to address the aforementioned issues of computational difficulty, high training complexity, and high retrieval complexity. In this research, a LSTM structure based improved methodology is proposed. The object's composition is shown in Fig. 4.

The dynamic time features of a neural network are expressed via a directed loop known as a recurrent neural network (RNN). The vanishing gradient problem, which states that as the amount of network layers rises, the gradient fades layer by layer, compromises the effectiveness of the weight adjustment process, is a drawback of the gradient descent approach employed in RNN. When processing and predicting lengthy sequences with event lags and erratic timing, a cyclic neural network performs worse than the LSTM model, a kind of RNN. The gradient dissipation issue is resolved by the Recurrent Projection Layer that LSTM adds to the RNN model. When the parameters are transferred backward in the Recurrent Projection Layer, the method presented in this paper also uses a dropout module to arbitrarily omit specific parameter values.

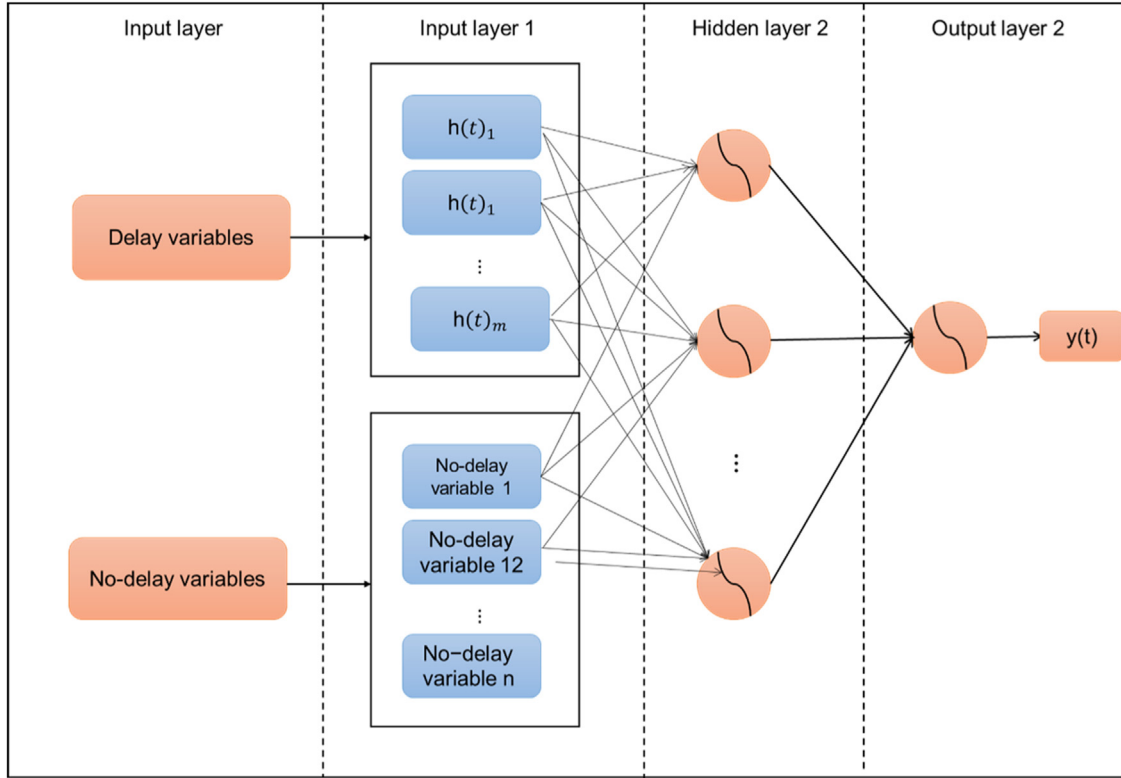


Fig. 3. LSTM model uses the Hidden Layer (HL) node in the recurrent neural network (RNN).

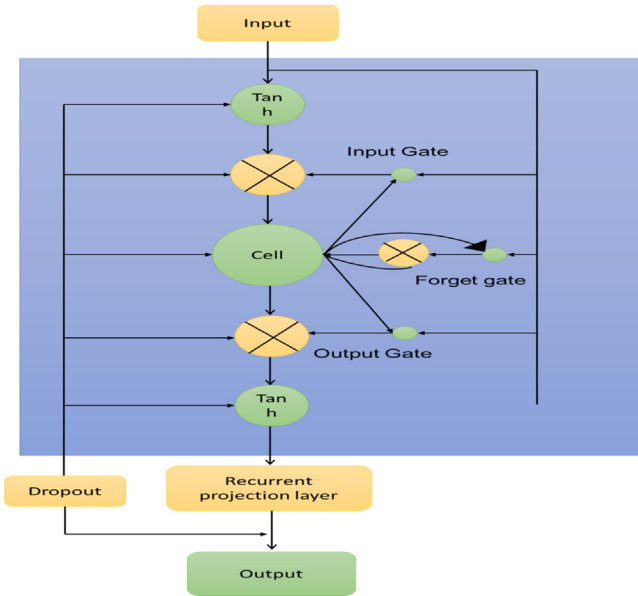


Fig. 4. Improved LSTM model.

This method meets the quick detection criteria of real-time complex retrieval while reducing the training resource requirements.

The backpropagation technique in Fig. 4 must also be modified to account for this upgraded model. Eq. (8) represents the loss function, which solely accounts for the output at the most recent time step:

$$L_N = \frac{1}{2} \sum_{i=1}^N (O_k^1 - y_k^1)^2 \quad (8)$$

The first and second steps are the same as those in Fig. 4. Eq. (9) should be written as follows to represent Step 3.

$$\sigma(u_{k,1}^1) = \sigma([Wh_k^1 W^n r^1] + [bb^n]) \quad (9)$$

Where Eq. (11) represents r_i stands for the weight w^n along with the b^n corresponding bias matrices for the no-delay variables, where $u_{k,1}^1$ signifies the output of HL 1, which is not enabled. As a result of the new HL 2 Eqs. (10), (11), the next two stages are written as follows.

$$o_k^1 = \sigma(u_{k,2}^1) = \sigma(W^{out} \sigma(u_{k,1}^1) + b^{out}) \quad (10)$$

$$\delta_k^1 = \sigma(o_k^1 - y_k^1) \odot \sigma'(u_{k,2}^1) \quad (11)$$

Where $u_{k,2}^1$ indicates the output of hidden layer 2, which is not activated, w^{out} and b^{out} represents the weight and bias matrices of the output layer of Eqs. (12) to (24) in the manner shown. The subsequent steps are stated as follows:

$$\delta_{o,k}^1 = (W^{out} W)^T \delta_k^1 \odot \sigma'(u_{k,2}^1) \odot \tanh(s_k^1) \odot \sigma'(u_{o,k}^1) \quad (12)$$

$$\delta_{f,k}^1 = (W^{out} W)^T \delta_k^1 \odot \sigma'(u_{k,1}^1) \odot \tanh'(s_k^1) \odot \sigma'(u_{i,k}^1) \odot s_{k-1}^1 \odot \sigma(u_{o,k}^1) \quad (13)$$

$$\delta_{i,k}^1 = (W^{out} W)^T \delta_k^1 \odot \sigma'(u_{i,k}^1) \odot \tanh'(s_k^1) \odot \tanh'(u_{g,k}^1) \odot i_k^1 \odot \sigma(u_{o,k}^1) \quad (14)$$

$$\delta_{g,k}^1 = (W^{out} W)^T \delta_k^1 \odot \sigma'(u_{g,k}^1) \odot \tanh'(s_k^1) \odot \tanh'(u_{g,k}^1) \odot i_k^1 \odot \sigma(u_{o,k}^1) \quad (15)$$

$$\frac{\partial L_N}{\partial W^{out}} = \sum_{l=1}^N \delta_k^l \left(\sigma(u_{g,k}^l) \right)^T \quad (16)$$

$$\frac{\partial L_N}{\partial W^{out}} = \sum_{l=1}^N \delta_k^l \quad (17)$$

$$\frac{\partial L_N}{\partial W^{out}} = \sum_{l=1}^N (W^{out})^T \delta_k^l \odot \sigma'(u_{k,l}^l) (h_k^l)^T \quad (18)$$

$$\frac{\partial L_N}{\partial W^n} = \sum_{l=1}^N (W^{out})^T \delta_k^l \odot \sigma'(u_{k,l}^l) (r^l)^T \quad (19)$$

$$\frac{\partial L_N}{\partial b} = \sum_{l=1}^N (W^{out})^T \delta_k^l \odot \sigma'(u_{k,l}^l) \quad (20)$$

$$\frac{\partial L_N}{\partial b^n} = \sum_{l=1}^N (W^{out})^T \delta_k^l \odot \sigma'(u_{k,l}^l) \quad (21)$$

$$\frac{\partial L_N}{\partial W^{x\tau}} = \sum_{l=1}^N \delta_{\tau,k}^l (x_k^l)^T \quad (22)$$

$$\frac{\partial L_N}{\partial W^{h\tau}} = \sum_{l=1}^N \delta_{\tau,k}^l (h_{k-1}^l)^T, \tau \in \{g, i, f, o\} \quad (23)$$

$$\frac{\partial L_N}{\partial b_\tau} = \sum_{l=1}^N \delta_{\tau,k}^l \quad (24)$$

The weights and biases must be updated as the final step.

4. Experimental results and discussion

According to MATLAB, the suggested approach to measuring the cyber security risks in IoT outperforms state-of-the-art algorithms in categorization.

The effectiveness of the ATFDNN+IPSO has been assessed and compared against other prominent DL methods, such as Restricted Boltzmann Machines (RBMs), Deep Belief Networks (DBNs), Deep Boltzmann Machines (DBMs), and Deep Auto-Encoders (DAs) Ferrag et al. [22]. This comparison highlights the efficiency of ATFDNN+IPSO in detecting malware-infected programs and illegal records within IoT settings.

4.1. Accuracy

The degree to which a measured value is accurate to reference or known value. The more common definition of measurement accuracy is the extent to which the result of a measurement agrees with the actual value of the thing being measured. As its name implies, the accuracy aspect of data quality refers to the information's accuracy. If you want to determine whether data is reliable, consider whether it depicts a situation that would occur.

Fig. 5 demonstrates that the proposed strategy -ATFDNN+IPSO (95%) outperformed the findings for the current methods of RBM (77%), DBN (84%), DBM (63%), and DA (55%) in a comparative examination of accuracy. It shows that the suggested system performs excellently.

4.2. False alarm ratio

The proportion of false alerts to all warnings or alarms in a certain study or circumstance is known as the false alarm ratio, or FAR for short. The false alarm ratio and false alarm rate are sometimes confused since they both share the same acronym (FAR). A comparison of the false alarm ratio is shown in Fig. 6.

Fig. 6 demonstrates that the proposed strategy-ATFDNN+IPSO (55%) outperformed the findings for the current methods of RBM (91%), DBN (65%), DBM (74%), and DA (81%) in a comparative examination of FAR. It shows that the suggested system performs excellently.

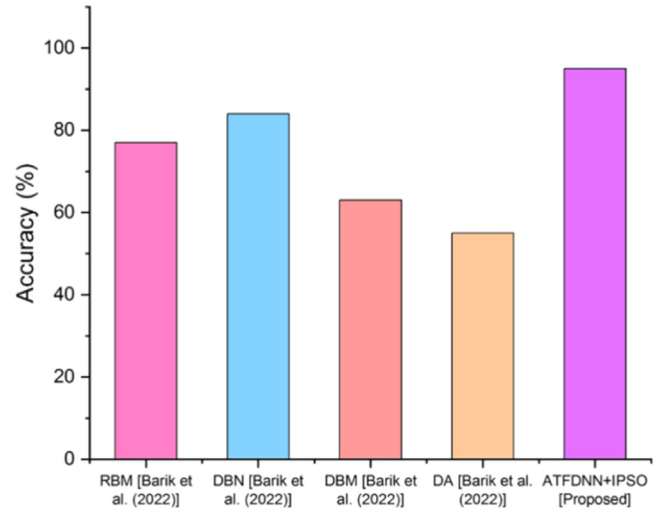


Fig. 5. Comparative analysis of accuracy.

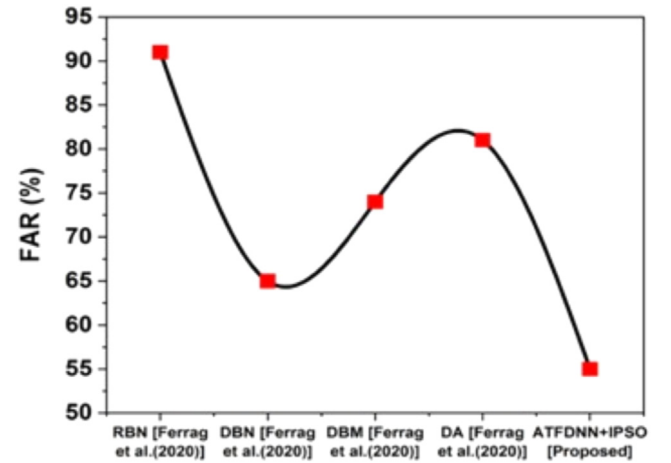


Fig. 6. Comparative analysis of false alarm ratio.

4.3. Training time

Training time includes all formally organized team-based communication and learning activities, as well as process-related periods (training on the job) and off-the-job periods (training in training facilities/factories or outside the company). There is a comparison of the Training in Fig. 7.

Fig. 7 demonstrates that the proposed strategy-ATFDNN+IPSO (50%) outperformed the findings for the current methods of RBM (74%), DBN (65%), DBM (88%), and DA (96%) in a comparative examination of Training time. It shows that the suggested system performs excellently.

4.4. Detection rate overall

The percentage of all patients that have the condition and are labeled positive by the diagnostic test is known as the detection rate. The percentage of patients that test positively is known as the recall rate. The difference between detection rate and count rate, which is the amount of a certain species detected in a given amount of time, is the frequency of hit events per laser pulse, which is characteristically on the order of one per 100 – 1000 laser pulses. Comparisons of the overall detection rate are shown in Fig. 8.

Fig. 8 demonstrates that the proposed strategy-ATFDNN+IPSO (98%) outperformed the findings for the current methods of RBM (53%), DBN (63%), DBM (74%), and DA (88%) in a comparative examination

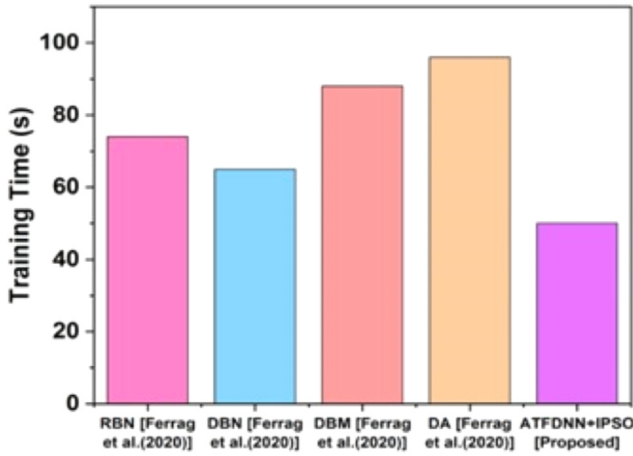


Fig. 7. Comparative analysis of Training Time.

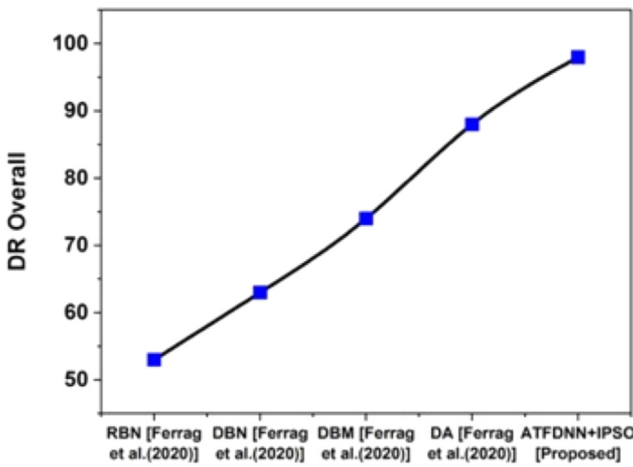


Fig. 8. Comparisons of detection rate overall.

of comparison of detection ratio overall. It shows that the suggested system performs excellently.

4.5. Ablation experiment

An ablation experiment is used to examine the relative contributions of different parts or qualities inside a system. By carefully removing or altering these components, researchers can determine the impact that these pieces have on the system's overall performance. This makes it easier to understand how each component works and how important it is to achieving the system's objectives. To verify the effectiveness of the suggested method even more, an ablation experiment was carried out to separate the contributions of different system parts. According to this experiment, obtaining the high performance metrics of the suggested strategy requires both the ATFDNN and IPSO components. Lower detection rate, longer training time, higher FAR, and worse accuracy are the outcomes of removing either component.

4.5.1. Purpose of the ablation experiment

The primary goal of the ablation experiment is to evaluate how each component of the proposed approach (ATFDNN+IPSO) contributes to its performance metrics, such as accuracy, false alarm ratio (FAR), training time, and detection rate. This process helps identify which elements are critical for the system's success and which are less influential.

4.5.2. Ablation experiment procedure

The Procedure of the Ablation Experiment for the proposed method of this study was given by the following steps.

- 1. Baseline System:** Start with the entire suggested system, ATFDNN+IPSO, and assess its effectiveness using a range of metrics.
- 2. Part Elimination:**
 - Without ATFDNN: Remove the ATFDNN part of the system and assess how well it functions. This facilitates the assessment of the contribution of ATFDNN to the detection rate, accuracy, FAR, and training time.
 - Without IPSO: Remove the IPSO element and assess how well it works. This evaluates how IPSO affects the metrics of the system.
 - Without Both ATFDNN and IPSO: Disable both and assess performance to determine the overall impact of disabling both system components.
- 3. Comparison:** Examine and contrast the system's overall performance metrics with those derived from the ablated versions. This comparison highlights the significance and potency of every element. The outcomes of the ablation experiment are reported in Table 1 and Fig. 9.

The ablation study results offer insightful information on how the Improved Particle Swarm Optimization (IPSO) and Adaptive Tensor-Flow Deep Neural Network (ATFDNN) contribute to the overall performance of the model. Using 95% accuracy, 55% false alarm ratio (FAR), 50% training time, and 98% detection rate, the complete system using ATFDNN and IPSO attained the top metrics. When ATFDNN was removed, accuracy significantly decreased to 89%, FAR increased to 65%, and training time to 60% and detection rate dropped to 91%, indicating the critical role ATFDNN plays in improving accuracy and lowering false alarms. While accuracy fell to 92% and FAR increased to 60%, Training Time slightly decreased to 55% and detection rate 95%, training time rose to 55% when IPSO was eliminated, the impact was not as great as it was when ATFDNN was deleted. Finally, in the absence of both ATFDNN and IPSO, the model's detection rate decreased to 87%, FAR rate to 70%, and accuracy dropped to 85% and training time escalated to 65%, demonstrating the necessity of both elements for peak performance. The lowest performance measurements are obtained by removing both components. This emphasizes how crucial ATFDNN and IPSO are when paired.

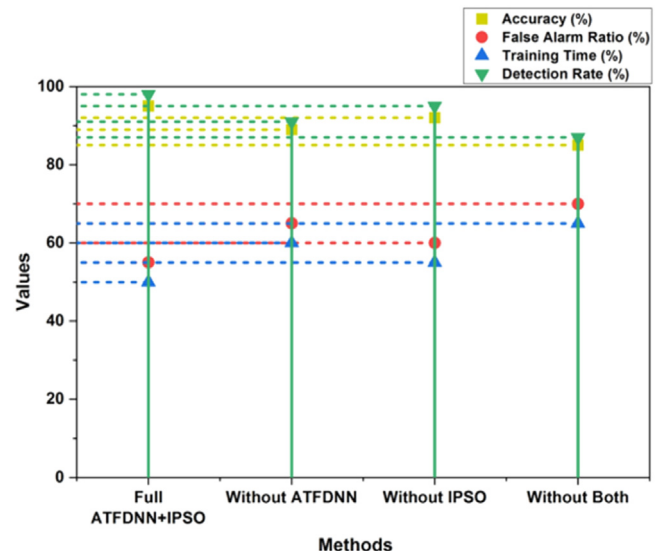


Fig. 9. Ablation experiment.

Table 1
Outcome of ablation experiment.

Method	Accuracy (%)	False Alarm Ratio (%)	Training Time (%)	Detection Rate (%)
Full ATFDNN+IPSO	95	55	50	98
Without ATFDNN	89	65	60	91
Without IPSO	92	60	55	95
Without Both	85	70	65	87

4.6. Discussion

The RBM's weight adjustment process is not as accurate as the back-propagation algorithms. It is difficult to calculate the energy gradient function, which makes training the constrained Boltzmann machine challenging. DBN has complicated data models, which makes training expensive. The single bottom-up pass of Deep Belief Networks is faster than DBM mean field-based approximation inference, which is a downside. The primary problem with DA is that their encoded vectors' locations and the latent space to which they transform their inputs may not be continuous or amenable to straightforward interpolation. Therefore Asymmetric tensor flow deep neural network with improved particle swarm optimization displays stability and is unaffected by a wide range of inputs. It can be combined and applied with several other strategies to produce even better outcomes. These existing methods give lower performance compared to the proposed method. To overcome this, we have proposed ATFDNN with IPSO.

By adding sophisticated features like attention mechanisms and optimized gate designs, enhanced long-short term memory (ELSTM) outperforms long-short term memory (LSTM) and produces sequence predictions that are more accurate. This improves the model's performance, especially for challenging jobs like cybersecurity threat detection where it's essential to recognize patterns precisely.

To identify suspicious actions in the IoT environment, the Enhanced Long Short-Term Memory (E-LSTM) model utilizes visual representations of color images as a key input. By processing these images, E-LSTM captures intricate visual features and patterns over time, which is critical for detecting anomalies and unusual behaviors. The integration of attention mechanisms within E-LSTM further refines this process by focusing on significant visual cues and temporal dependencies, enhancing the model's ability to discern subtle and complex patterns. This sophisticated approach not only improves the accuracy of threat detection but also ensures timely and precise responses to potential security breaches.

5. Conclusion

When leveraging the IoT-based big data, the detection of malware threats and software piracy are the key concerns in the realm of cybersecurity. Here, we provide a hybrid DL approach for locating malicious software and data that have been unintentionally disseminated throughout the Internet of Things (IoT) ecosystem. The Adaptive Tensor Flow Deep Neural Network (ATFDNN) with Improved Particle Swarm Optimization (IPSO) is recommended for SC duplication-based unlawful content detection. We obtained 100 SC files from programmers from GCJ to research the suggested approach. The trial results show that the combined approach returns the greatest level of classification results when compared to cutting-edge methods. The black box character of adaptive sensor flow deep neural networks, the added computing load, the susceptibility to overfitting, and the empirical nature of the model building are their disadvantages. Particle swarm optimization (PSO) has a modest rate of convergence during the iterative phase, making it easy to enter a local optimum in high-dimensional space.

A practical limitation of the proposed hybrid ATFDNN+IPSO strategy is its reliance on large, high-quality datasets, such as those from Google Code Jam (GCJ) and Maling, for effective malware and software piracy detection. In real-world IoT environments, data may be incomplete, noisy, or imbalanced, which could affect the model's accuracy and

generalizability. Additionally, the computational cost associated with training deep neural networks, especially with optimization methods like IPSO, may pose challenges for resource-constrained IoT devices, impacting real-time threat detection. Future research may also examine the potential for hybrid traits, which combine static and dynamic qualities, for malware identification.

Data availability

Not applicable.

Consent for publication

Not applicable.

Funding

The authors did not receive financial support from any organization for the submitted work.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

S. Markkandeyan: Supervision, Formal analysis. **A. Dennis Ananth:** Project administration, Methodology. **M. Rajakumaran:** Writing – original draft, Validation, Project administration. **R.G. Gokila:** Data curation, Conceptualization. **R. Venkatesan:** Writing – review & editing, Resources. **B. Lakshmi:** Software, Resources.

References

- [1] M. Sewak, S.K. Sahay, H. Rathore, Deep reinforcement learning for cybersecurity threat detection and protection: a review, in: Proceedings of the Secure Knowledge Management In The Artificial Intelligence Era, 9th International Conference, SKM 2021, Springer International Publishing, 2021, pp. 51–72. ChamOctober.
- [2] P. Sornsuwit, S. Jaiyen, A new hybrid machine learning for cybersecurity threat detection based on adaptive boosting, Appl. Artif. Intell. 33 (2019) 462–482.
- [3] J. Lee, J. Kim, I. Kim, K. Han, Cyber threat detection based on artificial neural networks using event profiles, IEEE Access 7 (2019) 165607–165626.
- [4] K.R. Dalal, M. Rele, C. Security, Threat detection model based on machine learning algorithm, in: Proceedings of the 3rd International Conference on Communication and Electronics Systems (ICCSES), IEEE, 2018, pp. 239–243. October.
- [5] W.H. Lin, H.C. Lin, P. Wang, B.H. Wu, J.Y. Tsai, Using convolutional neural networks to network intrusion detection for cyber threats, in: Proceedings of the IEEE International Conference on Applied System Invention (ICASI), IEEE, 2018, pp. 1107–1110. April.
- [6] K. Thakur, M. Qiu, K. Gai, M.L. Ali, An investigation on cyber security threats and security models, in: Proceedings of the 2nd International Conference on Cyber Security and Cloud Computing, IEEE, 2015, pp. 307–311. November.
- [7] F. Ullah, H. Naem, S. Jabbar, S. Khalid, M.A. Latif, F. Al-Turjman, L. Mostarda, Cyber security threats detection in internet of things using deep learning approach, IEEE Access 7 (2019) 124379–124389.
- [8] K. Barik, S. Misra, K. Konar, L. Fernandez-Sanz, M. Koyuncu, Cybersecurity deep: approaches, attacks dataset, and comparative study, Appl. Artif. Intell. 36 (1) (2022) 2055399.
- [9] N.M. Karie, N.M. Sahri, P. Haskell-Dowland, IoT threat detection advances, challenges and future directions, in: 2020 Workshop On Emerging Technologies For Security in IoT (ETSecIoT), IEEE, 2020, pp. 22–29. April.

- [10] I.H. Sarker, Y.B. Abushark, F. Alsolami, A.I. Khan, Intrudtree: a machine learning based cyber security intrusion detection model, *Symmetry* 12 (2020) 754 (Basel).
- [11] K.H. Le, M.H. Nguyen, T.D. Tran, N.D. Tran, IMIDS: an intelligent intrusion detection system against cyber threats in IoT, *Electronics* 11 (2022) 524 (Basel).
- [12] X.C. Yin, Z.G. Liu, L. Nkenyereye, B. Ndibanje, Toward an applied cyber security solution in IoT-based smart grids: an intrusion detection system approach, *Sensors* 19 (2019) 4952.
- [13] I.F. Kilincer, F. Ertam, A. Sengur, Machine learning methods for cyber security intrusion detection: datasets and comparative study, *Comput. Netw.* 188 (2021) 107840.
- [14] H. Alqahtani, I.H. Sarker, A. Kalim, S.M. Minhaz Hossain, S. Ikhlal, S. Hossain, Cyber intrusion detection using machine learning classification techniques, in: *Proceedings of the Computing Science, Communication and Security: First International Conference, COMS2*, Singapore, Springer, 2020, pp. 121–131.
- [15] I.A. Kandhro, S.M. Al Enezi, F. Ali, A. Kehar, K. Fatima, M. Uddin, S. Karuppayah, Detection of real-time malicious intrusions and attacks in IoT empowered cyber security infrastructures, *IEEE Access* 11 (2023) 9136–9148.
- [16] T.H. Aldhyani, H. Alkahtani, Cyber security for detecting distributed denial of service attacks in agriculture 4.0: deep learning model, *Mathematics* 11 (2023) 233.
- [17] V.O. Kayhan, M. Agrawal, S. Shivendu, Cyber threat detection: unsupervised hunting of anomalous commands (UHAC), *Decis. Support Syst.* 168 (2023) 113928.
- [18] M. Ahsan, K.E. Nygard, R. Gomes, M.M. Chowdhury, N. Rifat, J.F. Connolly, Cyber-security threats and their mitigation approaches using machine learning - A review, *J. Cybersecur. Priv.* 2 (2022) 527–555.
- [19] M. Al Razib, D. Javeed, M.T. Khan, R. Alkanhel, M.S.A. Muthanna, Cyber threats detection in smart environments using SDN-enabled DNN-LSTM hybrid framework, *IEEE Access* 10 (2022) 53015–53026.
- [20] P.M. Laso, L. Salmon, M. Bozhilova, I. Ivanov, N. Stoianov, G. Velez, C. Claramunt, Y. Yanakiev, ISOLA: an innovative approach to cyber threat detection in cruise shipping, in: *developments and advances in defense and security*, in: *Proceedings of the MICRADS*, Singapore, Springer, 2021, pp. 71–81.
- [21] D. Pant, R. Bista, Image-based malware classification using deep convolutional neural network and transfer learning, in: *Proceedings of the 3rd International Conference on Advanced Information Science and System, AISS*, 2021, pp. 1–6. 2021November.
- [22] M.A. Ferrag, L. Maglaras, S. Moschogiannis, H. Janicke, Deep learning for cyber security intrusion detection: approaches, datasets, and comparative study, *J. Inf. Secur. Appl.* 50 (2020) 102419.